AI ASSISTED CODING

LAB ASSIGNMENT-2

NAME: M. Shashi

HTNO: 2303A51028

Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI

Task 1: Statistical Summary for Survey Data

Scenario:

You are a data analyst intern working with survey responses stored as numerical lists.

Task:

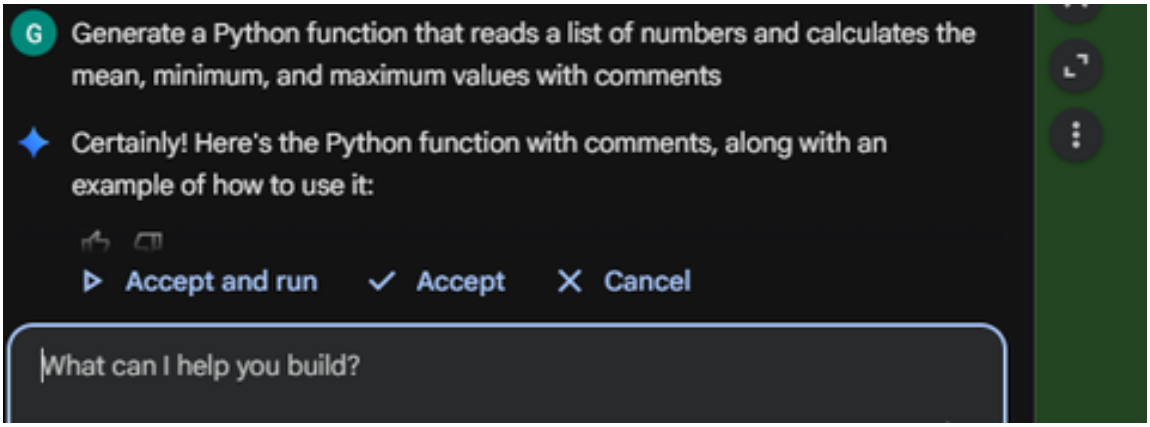
Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

Expected Output:

➢ Correct Python function

➢ Output shown in Colab

➢ Screenshot of Gemini prompt and result



```python
import statistics # Import the statistics module for mean calculation
def analyze_numbers_with_comments(numbers):
  """
  Calculates the mean, minimum, and maximum values from a list of numbers.
  Args:
    numbers: A list of numerical values (e.g., integers or floats).
  Returns:
    A tuple containing (mean, minimum, maximum).
    Returns (None, None, None) if the input list is empty.
  """
  # Check if the list is empty to prevent errors
  if not numbers:
    return None, None, None
  # Calculate the mean using the statistics module
  mean_value = statistics.mean(numbers)
  # Find the minimum value in the list
  min_value = min(numbers)
  # Find the maximum value in the list
  max_value = max(numbers)
  # Return the calculated values as a tuple
  return mean_value, min_value, max_value
# --- Example Usage ---
# Define a sample list of numbers
sample_numbers = [5, 15, 25, 35, 45, 10]
# Call the function to analyze the list
mean, minimum, maximum = analyze_numbers_with_comments(sample_numbers)
# Print the original list and the calculated results
print(f"List of numbers: {sample_numbers}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")
# --- Example with an empty list to demonstrate error handling ---
empty_list = []
mean, minimum, maximum = analyze_numbers_with_comments(empty_list)
print(f"\nList of numbers: {empty_list}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")
```

```
List of numbers: [5, 15, 25, 35, 45, 10]
Mean: 22.5
Minimum: 5
Maximum: 45

List of numbers: []
Mean: None
Minimum: None
Maximum: None
```

Task 2: Armstrong Number – AI Comparison

❖ Scenario:

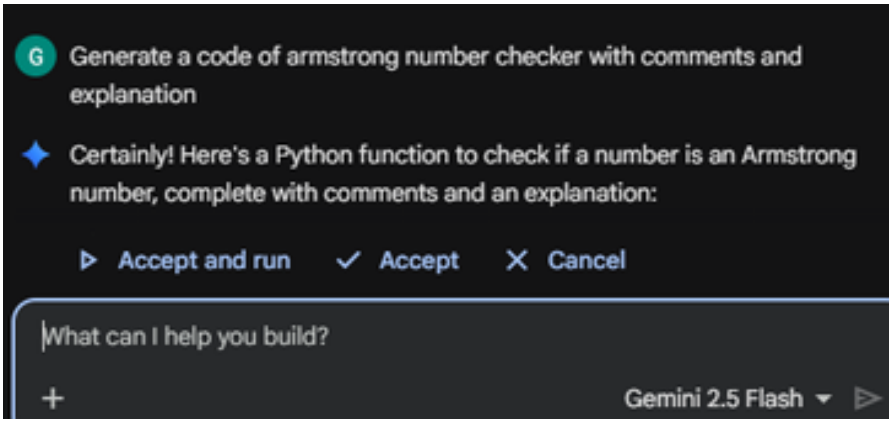
You are evaluating AI tools for numeric validation logic.

❖ Task:

Generate an Armstrong number checker using Gemini and GitHub Copilot.

Compare their outputs, logic style, and clarity.

Gemini copilot:



```python
def is_armstrong_number(number):
  """
  Checks if a given number is an Armstrong number.
  An Armstrong number (also known as a narcissistic number, plenary number,
  or plus perfect number) is a number that is the sum of its own digits
  each raised to the power of the number of digits.
  For example, 153 is an Armstrong number because 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.
  Args:
    number: An integer to check.
  Returns:
    True if the number is an Armstrong number, False otherwise.
  """
  # Convert the number to a string to easily get its digits and count them
  num_str = str(number)
  # Get the number of digits
  num_digits = len(num_str)
  # Initialize the sum of powers of digits
  sum_of_powers = 0

  # Iterate through each digit in the number string
  for digit_char in num_str:
    # Convert the character digit back to an integer
    digit = int(digit_char)
    # Add the digit raised to the power of num_digits to the sum
    sum_of_powers += digit ** num_digits

  # Compare the original number with the calculated sum
  return number == sum_of_powers
# --- Example Usage ---
print("--- Armstrong Number Checker ---")
# Test with known Armstrong numbers
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}") # Expected: True
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}") # Expected: True
print(f"Is 371 an Armstrong number? {is_armstrong_number(371)}") # Expected: True
print(f"Is 407 an Armstrong number? {is_armstrong_number(407)}") # Expected: True

# Test with a non-Armstrong number
print(f"Is 123 an Armstrong number? {is_armstrong_number(123)}") # Expected: False
print(f"Is 1634 an Armstrong number? {is_armstrong_number(1634)}") # Expected: True
print(f"Is 100 an Armstrong number? {is_armstrong_number(100)}") # Expected: False
```

```
--- Armstrong Number Checker ---
Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 371 an Armstrong number? True
Is 407 an Armstrong number? True
Is 123 an Armstrong number? False
Is 1634 an Armstrong number? True
Is 100 an Armstrong number? False
```

Github copilot

Generate a code of armstrong number checker with inline comments and explanation

Add Context...

```python
# Armstrong Number Checker
# An Armstrong number is a number that is equal to the sum of its own digits each raised to the power of the number of digits

def is_armstrong(num):
    # Convert number to string to easily access individual digits
    num_str = str(num)

    # Get the number of digits
    num_digits = len(num_str)

    # Calculate the sum of each digit raised to the power of total digits
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
    # Check if the sum equals the original number
    return sum_of_powers == num
# Test the function
if __name__ == "__main__":
    test_numbers = [153, 370, 371, 407, 1634, 100, 9474]

    for num in test_numbers:
        if is_armstrong(num):
            print(f"{num} is an Armstrong number")
        else:
            print(f"{num} is not an Armstrong number")
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> &
153 is an Armstrong number
370 is an Armstrong number
371 is an Armstrong number
407 is an Armstrong number
1634 is an Armstrong number
100 is not an Armstrong number
9474 is an Armstrong number
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Aspect	GitHub Copilot Output	Gemini Output
Overall Style	Short and concise, focuses on direct logic	More detailed and explanatory
Logic Implementation	Uses Python's sum() with a generator expression	Uses a loop to calculate the sum step by step
Readability	Compact but may feel slightly advanced for beginners	Very beginner-friendly and easy to follow
Use of Comments	Fewer comments, assumes basic Python knowledge	Many inline comments explaining each step
Clarity of Flow	Logic is correct but condensed into fewer lines	Logic is broken into small, clear steps
Beginner Suitability	Better for users comfortable with Python syntax	Better for beginners learning number logic
Maintainability	Easy to maintain but less descriptive	Easier to modify and debug due to clarity

Task 3: Leap Year Validation Using Cursor AI

❖ Scenario:

You are validating a calendar module for a backend system.

❖ Task:

Use Cursor AI to generate a Python program that checks whether a given

year is a leap year.

Use at least two different prompts and observe changes in code.

❖ Expected Output:

➢ Two versions of code

➢ Sample inputs/outputs

➢ Brief comparison

Prompt 1:

Python leap year program

2m · +52 · Auto

generate a Python program that checks whether a given year is a leap year with comments

leap_year_checker.py

```python
"""
Leap Year Checker Program
This program checks whether a given year is a leap year.
A leap year follows these rules:
1. If the year is divisible by 400, it is a leap year
2. Else if the year is divisible by 100, it is NOT a leap year
3. Else if the year is divisible by 4, it is a leap year
4. Otherwise, it is NOT a leap year
"""
def is_leap_year(year):
    """
    Check if a given year is a leap year.
    Args:
        year (int): The year to check
    Returns:
        bool: True if the year is a leap year, False otherwise
    """
    # Rule 1: Years divisible by 400 are always leap years
    # This handles century years like 2000, 2400
    if year % 400 == 0:
        return True
    # Rule 2: Years divisible by 100 but not by 400 are NOT leap years
    # This handles century years like 1900, 1800, 1700
    if year % 100 == 0:
        return False
    # Rule 3: Years divisible by 4 are leap years
    # This handles regular leap years like 2004, 2008, 2020
    if year % 4 == 0:
        return True
    # Rule 4: All other years are NOT leap years
    return False
def main():
    """
    Main function to demonstrate the leap year checker.
    """
```

```python
def main():
    """
    Main function to get runtime input and check if year is a leap year.
    """
    # Get input from user at runtime
    try:
        year = int(input("Enter a year to check: "))
        # Check if the year is a leap year
        if is_leap_year(year):
            print(f"{year} is a leap year.")
        else:
            print(f"{year} is not a leap year.")
    except ValueError:
        # Handle invalid input (non-numeric values)
        print("Error: Please enter a valid integer year.")
# Run the program if executed directly
if __name__ == "__main__":
    main()
```

```
/varsh/OneDrive/ドキュメント/third/AIAC/leap_year_checker.py
Enter a year to check: 2024
2024 is a leap year.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/v
arsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users
/varsh/OneDrive/ドキュメント/third/AIAC/leap_year_checker.py
Enter a year to check: 2004
2004 is a leap year.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Prompt 2:

6m · +136 · Auto

Generate code to check leap year from given input improving the previous code

1. Input validation — `validate_year()` checks:

leap_year_checker.py > validate_year

```python
"""
Leap Year Checker Program
This program checks whether a given year is a leap year using Gregorian calendar rules.
A leap year follows these rules:
1. If the year is divisible by 400, it is a leap year
2. Else if the year is divisible by 100, it is NOT a leap year
3. Else if the year is divisible by 4, it is a leap year
4. Otherwise, it is NOT a leap year
"""


def is_leap_year(year):
    """
    Check if a given year is a leap year using Gregorian calendar rules.

    Args:
        year (int): The year to check

    Returns:
        bool: True if the year is a leap year, False otherwise
    """
    # Check leap year using Gregorian calendar rules with clear conditions

    # Rule 1: Years divisible by 400 are always leap years
    # This handles century years like 2000, 2400, 1600
    if year % 400 == 0:
        return True

    # Rule 2: Years divisible by 100 but not by 400 are NOT leap years
    # This handles century years like 1900, 1800, 1700
    if year % 100 == 0:
        return False

    # Rule 3: Years divisible by 4 are leap years
    # This handles regular leap years like 2004, 2008, 2020, 2024
    if year % 4 == 0:
        return True

    # Rule 4: All other years are NOT leap years
    return False


def validate_year(year):
    """
    Validate if the year is a reasonable input.

    Args:
        year (int): The year to validate

    Returns:
        tuple: (bool, str) - (is_valid, error_message)
    """
```

```python
def validate_year(year):
    Returns:
        tuple: (bool, str) - (is_valid, error_message)
    """
    # Check if year is positive (Gregorian calendar starts from year 1)
    if year < 1:
        return False, "Year must be a positive number (>= 1)."

    # Check if year is within reasonable range (optional validation)
    if year > 10000:
        return False, "Year is too large. Please enter a year <= 10000."

    return True, ""


def get_leap_year_explanation(year):
    """
    Get a detailed explanation of why a year is or isn't a leap year.

    Args:
        year (int): The year to explain

    Returns:
        str: Explanation string
    """
    if year % 400 == 0:
        return f"{year} is divisible by 400, so it is a leap year."
    elif year % 100 == 0:
        return f"{year} is divisible by 100 but not by 400, so it is NOT a leap year."
    elif year % 4 == 0:
        return f"{year} is divisible by 4 (and not by 100), so it is a leap year."
    else:
        return f"{year} is not divisible by 4, so it is NOT a leap year."


def main():
    """
    Main function to get runtime input and check if year is a leap year.
    """
    print("=" * 60)
    print("Leap Year Checker - Gregorian Calendar Rules")
    print("=" * 60)

    # Get input from user at runtime
    try:
        # Get year input from user
        year_input = input("\nEnter a year to check: ").strip()

        # Validate that input is not empty
        if not year_input:
            print("Error: Please enter a year.")
            return
```

```python
def main():

    # Get input from user at runtime
    try:
        # Get year input from user
        year_input = input("\nEnter a year to check: ").strip()

        # Validate that input is not empty
        if not year_input:
            print("Error: Please enter a year.")
            return

        # Convert input to integer
        year = int(year_input)

        # Validate the year value
        is_valid, error_message = validate_year(year)
        if not is_valid:
            print(f"Error: {error_message}")
            return

        # Check leap year using Gregorian calendar rules with clear conditions
        if is_leap_year(year):
            print(f"\n✓ Result: {year} is a leap year.")
        else:
            print(f"\n✗ Result: {year} is not a leap year.")

        # Provide detailed explanation
        explanation = get_leap_year_explanation(year)
        print(f"Explanation: {explanation}")

    except ValueError:
        # Handle invalid input (non-numeric values)
        print("Error: Please enter a valid integer year (e.g., 2024, 1900, 2000).")

    except KeyboardInterrupt:
        # Handle Ctrl+C gracefully
        print("\n\nProgram interrupted by user.")

    except Exception as e:
        # Handle any other unexpected errors
        print(f"An unexpected error occurred: {e}")


# Run the program if executed directly
if __name__ == "__main__":
    main()
```

```
Enter a year to check: 2303

✗ Result: 2303 is not a leap year.
Explanation: 2303 is not divisible by 4, so it is NOT a leap year.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/
ve/ドキュメント/third/AIAC/leap_year_checker.py
============================================================
Leap Year Checker - Gregorian Calendar Rules
============================================================

Enter a year to check: 2004

✓ Result: 2004 is a leap year.
Explanation: 2004 is divisible by 4 (and not by 100), so it is a leap year.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/
ve/ドキュメント/third/AIAC/leap_year_checker.py
============================================================
Leap Year Checker - Gregorian Calendar Rules
============================================================

Enter a year to check: 2400

✓ Result: 2400 is a leap year.
Explanation: 2400 is divisible by 400, so it is a leap year.
Explanation: 2004 is divisible by 4 (and not by 100), so it is a leap year.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/
ve/ドキュメント/third/AIAC/leap_year_checker.py
============================================================
```

- **Logic Accuracy:**Version 1 uses a basic rule and may give incorrect results for century years.Version 2 follows proper Gregorian calendar rules and is fully accurate.
- **Code Quality:**Version 2 is more reliable and suitable for real-world backend systems.
- **Effect of Prompt Change:**A more detailed prompt resulted in better logic, correctness, and professional-quality code.

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ Scenario:

Company policy requires developers to write logic before using AI.

❖ Task:

Write a Python program that calculates the sum of odd and even numbers

in a tuple, then refactor it using any AI tool.

❖ Expected Output:

➢ Original code

➢ Refactored code

➢ Explanation of improvements

```python
t=tuple(map(int,input().split()))
odd_sum=0
even_sum=0
for i in t:
    if i%2==0:
        even_sum+=i
    else:
        odd_sum+=i
print("Odd sum : ",odd_sum)
print("Even sum: ",even_sum)
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/va
3 5 2 6 4 8
Odd sum :  8
Even sum:  20
```

Refactored Code:

CODE REFACTORING FOR READABILITY AND EFFICIENCY

Refactor this code to improve readability and effeiciency and also works for all test cases like negative

Lab_assignment-2.py:24-34

✓ Refactored code for improved readability and efficiency

✓ Edited Lab_assignment-2.py +0 -0

Done! Here's what I refactored:

Improvements made:

1. **Readability**: Added clear comments explaining the logic and handling of negative numbers
2. **Efficiency**: Replaced the manual loop with generator expressions inside `sum()` - this is more Pythonic and slightly faster
3. **Negative numbers**: Changed condition from `i%2==0` to `n%2!=0` for odd check (clearer logic). Python's modulo operator handles negatives correctly: `-1 % 2 = 1` (odd), `-2 % 2 = 0` (even)
4. **String formatting**: Used f-strings for cleaner output
5. **Variable naming**: Used `numbers` instead of `t` for clarity
6. **Removed commented code**: Cleaned up the unused Armstrong number code

The refactored code now:

- Works with positive, negative, and zero values
- Is more concise and readable
- Has better performance (no variable accumulation in loop)

leap_year_checker.py

Describe what to build next

```python
# Calculate sum of odd and even numbers from input
# Works with positive, negative, and zero values

numbers = tuple(map(int, input().split()))

# Calculate sums using generator expressions (efficient and readable)
# Works correctly with negative numbers due to Python's modulo behavior
odd_sum = sum(n for n in numbers if n % 2 != 0)
even_sum = sum(n for n in numbers if n % 2 == 0)

print(f"Odd sum : {odd_sum}")
print(f"Even sum: {even_sum}")
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL Python:

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/
on.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/Lab_assignment-2.py
1 7 -9 4 3 6 8
Odd sum : 2
Even sum: 18
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

What was improved?

The loop was replaced with Python’s built-in sum() function.The code became shorter and more readable. Redundant variable updates were removed.

The refactored code better.

Readability: Easier to understand at a glance.

Efficiency: Uses Python’s optimized built-in functions.

Maintainability: Fewer lines make the code easier to modify